Corso di Cybersecurity – Epicode

Report dell'Esercitazione

Analisi dell'handshake TCP a tre vie con Wireshark

Cattura con tcpdump e lettura dei flag, delle porte e dei numeri di sequenza

A cura di:

Nicola Madonna

Epicode

1 settembre 2026

Indice

1 Riassunto 2
2 Obiettivi 2
3 Configurazione del Laboratorio 3
3.1 Ambiente 3
3.2 Cattura del traffico 3
4 Analisi dei Dati Raccolti 4
4.1 Sequenza dei pacchetti 4
4.2 Frame 19: il SYN del client 4
4.3 Frame 20 e 21: completamento della connessione 5
4.4 Chiusura della connessione 5
4.5 Filtri utili in Wireshark 6
4.6 Usi in una rete di produzione 6
5 Conclusioni 7

1 Riassunto

In questo laboratorio è stata catturata con `tcpdump` una breve connessione tra un client e un server web e la cattura è stata analizzata in Wireshark. L'obiettivo era osservare pacchetto per pacchetto come TCP apre una connessione con l'handshake a tre vie (SYN, SYN-ACK, ACK) e come la chiude. Nella cattura la connessione tra il client `10.0.0.11` e il server `172.16.0.40` occupa i frame dal 19 al 24: i primi tre aprono la sessione, gli ultimi tre la chiudono con lo scambio dei FIN.

2 Obiettivi

- Catturare il traffico di una connessione HTTP con tcpdump e salvarlo su file.
- Riconoscere in Wireshark i tre pacchetti dell'handshake e i flag impostati in ciascuno.
- Leggere porte di origine e destinazione e numeri di sequenza e acknowledgment relativi.
- Individuare filtri di Wireshark e usi dello strumento utili in una rete di produzione.

3 Configurazione del Laboratorio

3.1 Ambiente

L'esercizio è stato svolto sulla macchina virtuale *CyberOps Workstation*, con una piccola rete emulata in cui l'host `H1` (client, `10.0.0.11`) raggiunge un server web all'indirizzo `172.16.0.40`. Sull'host H1 è stato aperto un browser verso il server, così da generare una connessione HTTP sulla porta 80.

3.2 Cattura del traffico

La cattura è stata avviata sull'interfaccia di H1 con il comando seguente, che salva su file i primi 50 pacchetti:

```
sudo tcpdump -i H1-eth0 -v -c 50 -w /home/analyst/capture.pcap
```

L'opzione `-i` sceglie l'interfaccia, `-v` attiva l'output dettagliato, `-c 50` ferma la cattura dopo 50 pacchetti e `-w` scrive i pacchetti grezzi nel file indicato. Al termine tcpdump conferma 50 pacchetti catturati e nessuno scartato dal kernel (Fig. 1).

```
sudo tcpdump -i H1-eth0 -v -c 50 -w /home/analyst/capture.p
cap
[sudo] password for analyst:
tcpdump: listening on H1-eth0, link-type EN10MB (Ethernet), snapshot length 2621
44 bytes
50 packets captured
50 packets received by filter
0 packets dropped by kernel
```

Figura 1: Cattura con tcpdump sull'interfaccia H1-eth0: 50 pacchetti catturati, nessuno scartato.

Il file salvato può essere riletto in seguito con `tcpdump -r capture.pcap`: l'opzione `-r` legge i pacchetti da un file di cattura invece di catturarli dal vivo da un'interfaccia. Per questa analisi il file è stato aperto in Wireshark.

4 Analisi dei Dati Raccolti

4.1 Sequenza dei pacchetti

Filtrando la cattura sulla connessione tra i due host restano 6 pacchetti su 50 (Fig. 2). I frame 19, 20 e 21 sono l'handshake di apertura; i frame 22, 23 e 24 sono la chiusura della connessione.

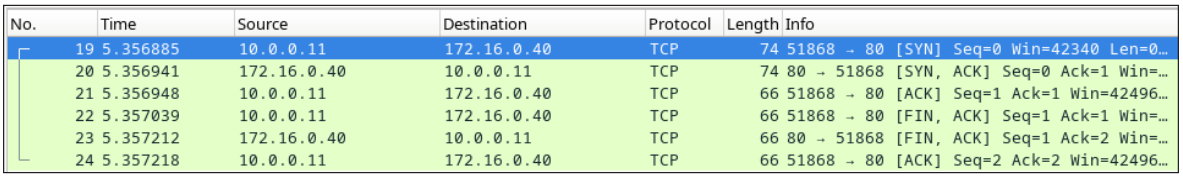


Figura 2: Frame 19–24 in Wireshark: apertura della connessione (SYN, SYN-ACK, ACK) e chiusura (FIN-ACK, FIN-ACK, ACK).

Tabella 1: Handshake di apertura e chiusura della connessione tra client e server.

Frame	**Direzione**	**Porte**	**Flag**	**Seq / Ack relativi**	**Significato**
19	client → server	51868 → 80	SYN	Seq=0, Ack=0	Il client chiede di aprire la connessione
20	server → client	80 → 51868	SYN, ACK	Seq=0, Ack=1	Il server conferma il SYN e invia il proprio
21	client → server	51868 → 80	ACK	Seq=1, Ack=1	Il client conferma: connessione stabilita
22	client → server	51868 → 80	FIN, ACK	Seq=1, Ack=1	Il client chiude la sua metà della connessione
23	server → client	80 → 51868	FIN, ACK	Seq=1, Ack=2	Il server conferma e chiude a sua volta
24	client → server	51868 → 80	ACK	Seq=2, Ack=2	Conferma finale: connessione chiusa

4.2 Frame 19: il SYN del client

Nel dettaglio del frame 19 (Fig. 3) la porta di origine è **51868** e quella di destinazione è **80**. La 51868 appartiene all'intervallo delle porte dinamiche o effimere (49152–65535), che il sistema operativo assegna automaticamente al client per le connessioni in uscita. La porta 80 è invece una porta *well-known* (0–1023), riservata al servizio HTTP.

Nel campo *Flags* (`0x002`) è impostato solo il bit **SYN**, che chiede la sincronizzazione dei numeri di sequenza per aprire una nuova connessione. Il numero di sequenza relativo è 0: Wireshark mostra i numeri relativi all'inizio del flusso per renderli leggibili, mentre quello reale (*raw*) è 3856717010. Nel SYN il client annuncia anche le proprie opzioni TCP: MSS di 1460 byte, supporto al SACK e timestamp.

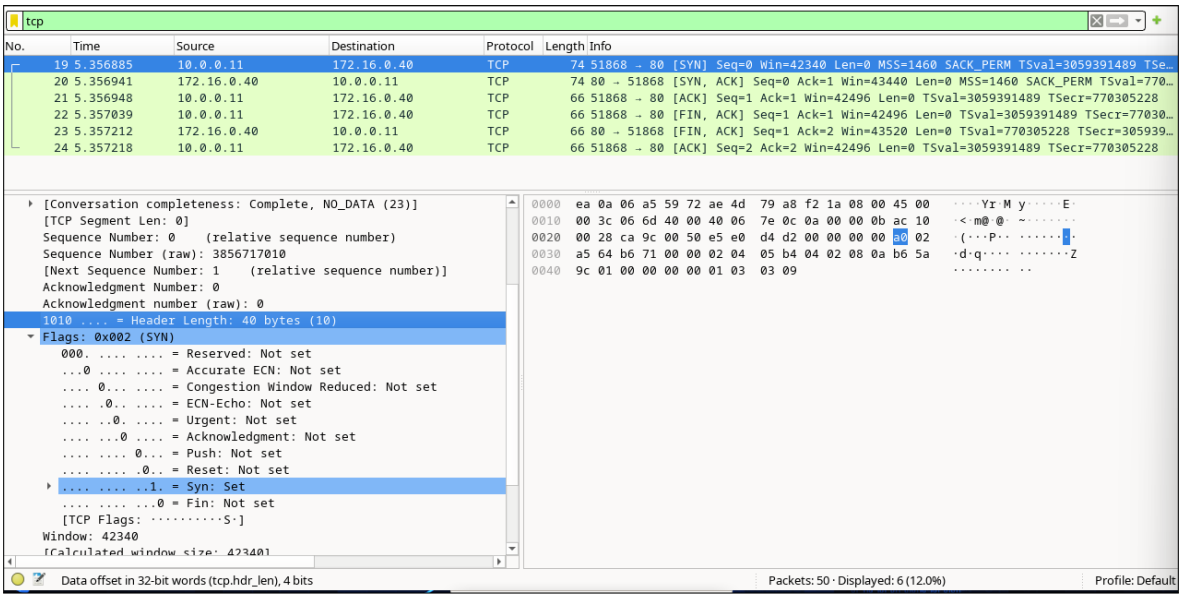


Figura 3: Dettaglio del frame 19: intestazione TCP con il solo flag SYN impostato e numero di sequenza relativo 0.

4.3 Frame 20 e 21: completamento della connessione

Nel frame 20 le porte sono invertite (origine 80, destinazione 51868) e sono impostati i flag **SYN** e **ACK**: il server conferma la richiesta e invia a sua volta il proprio SYN. Il numero di acknowledgment relativo vale 1, cioè il numero di sequenza del client più uno, perché il SYN occupa un numero di sequenza anche se non trasporta dati.

Nel frame 21 il client invia un pacchetto con il solo flag **ACK** (Seq=1, Ack=1) per confermare il SYN del server. A questo punto entrambe le parti hanno confermato i rispettivi numeri iniziali e la connessione è stabilita.

4.4 Chiusura della connessione

I frame 22–24 mostrano la chiusura: il client invia **FIN, ACK**, il server risponde con il proprio **FIN, ACK** (Ack=2, perché anche il FIN consuma un numero di sequenza) e il client chiude con un **ACK** finale. Anche la chiusura avviene quindi con tre pacchetti, perché il server ha unito la conferma del FIN del client e il proprio FIN in un unico segmento.

Tra l'apertura e la chiusura non ci sono pacchetti con dati: Wireshark classifica infatti la conversazione come *Complete, NO_DATA* (visibile in alto nella Fig. 3). La connessione è stata aperta e chiusa correttamente senza trasportare traffico HTTP, come accade ad esempio quando il browser apre una connessione in anticipo e poi non la utilizza.

4.5 Filtri utili in Wireshark

Tabella 2: Tre filtri di visualizzazione utili a un amministratore di rete.

Filtro	**A cosa serve**
`tcp.port == 80`	Isola il traffico HTTP da e verso un server web.
`ip.addr == 172.16.0.40`	Mostra tutto il traffico di un singolo host, utile in un'indagine mirata.
`tcp.flags.syn == 1 && tcp.flags.ack == 0`	Mostra solo i tentativi di apertura di connessione: un numero anomalo di SYN da una stessa sorgente può indicare un port scan o un SYN flood.

4.6 Usi in una rete di produzione

Oltre allo studio dei protocolli, in produzione Wireshark serve a diagnosticare problemi di prestazioni (ritrasmissioni, finestre TCP ridotte, latenze), a fare il debug di applicazioni che comunicano con protocolli poco documentati, a verificare la configurazione di VPN e traffico cifrato e, in ambito di sicurezza, ad analizzare il traffico durante la risposta a un incidente o nell'analisi forense successiva.

5 Conclusioni

La cattura ha mostrato in modo concreto il ciclo di vita di una connessione TCP: apertura con SYN, SYN-ACK e ACK e chiusura con lo scambio dei FIN, in questo caso senza dati applicativi nel mezzo. Leggere porte, flag e numeri di sequenza permette di capire chi ha iniziato la connessione, se è andata a buon fine e come è terminata.

Per un analista SOC questo è il punto di partenza per riconoscere anomalie: handshake lasciati a metà, molti SYN senza risposta o connessioni chiuse con RST sono spesso i primi segnali di una scansione o di un tentativo di attacco.